

```

$ ros2 bag record -h
usage: ros2 bag record [-h] [-a] [-o OUTPUT] [-s STORAGE]
                        [-f SERIALIZATION_FORMAT] [--no-discovery]
                        [-p POLLING_INTERVAL] [-b MAX_BAG_SIZE]
                        [topics [topics ...]]

ros2 bag record

positional arguments:
  topics                topics to be recorded

optional arguments:
  -h, --help            show this help message and exit
  -a, --all             recording all topics, required if no topics are listed
                        explicitly.
  -o OUTPUT, --output OUTPUT
                        destination of the bagfile to create, defaults to a
                        timestamped folder in the current directory
  -s STORAGE, --storage STORAGE
                        storage identifier to be used, defaults to "sqlite3"
  -f SERIALIZATION_FORMAT, --serialization-format SERIALIZATION_FORMAT
                        rmw serialization format in which the messages are
                        saved, defaults to the rmw currently in use
  --no-discovery        disables topic auto discovery during recording: only
                        topics present at startup will be recorded
  -p POLLING_INTERVAL, --polling-interval POLLING_INTERVAL
                        time in ms to wait between querying available topics
                        for recording. It has no effect if --no-discovery is
                        enabled.
  -b MAX_BAG_SIZE, --max-bag-size MAX_BAG_SIZE
                        maximum size in bytes before the bagfile will be
                        split. Default it is zero, recording written in single
                        bagfile and splitting is disabled.

```

We can see from the help file that the syntax for recording a bag is to simply give the sub command a list of topics to record. Most of the other arguments are for more advanced users to help configure how and when data is stored. It is worth noting that there is a `-a, --all` command that records all the data. You can also specify the output bag file with the `-o, --output` command.

Let's go ahead and run our bag command, and let's bag the pose data on the `/turtle1/pose` topic and save it to the file `turtle1.bag` using the `-o` flag. Be aware that the program will continue bagging data until you hit `CTRL-C`, so give the command a good 30 seconds to collect data before you kill it.

```
$ ros2 bag record /turtle1/pose -o turtle1
[INFO] [rosbag2_storage]: Opened database 'turtle1'.
[INFO] [rosbag2_transport]: Listening for topics...
[INFO] [rosbag2_transport]: Subscribed to topic '/turtle1/pose'
[INFO] [rosbag2_transport]: All requested topics are subscribed. Stopping
discovery...
^C[INFO] [rclcpp]: signal_handler(signal_value=2)
```

Now that we collected our data let's inspect our bag file. You can introspect any bag file using the `ros2 bag info` command. This command will list the messages in the bag, the duration of file, and the number of messages.

```
$ ros2 bag info turtle1
Files:          turtle1.db3
Bag size:       268.4 KiB
Storage id:     sqlite3
Duration:       68.705s
Start:          May  4 2020 16:10:26.556 (1588633826.556)
End:            May  4 2020 16:11:35.262 (1588633895.262)
Messages:       4249
Topic information: Topic: /turtle1/pose | Type: turtlesim/msg/Pose | Count: 4249 |
Serialization Format: cdr
```

Once you have collected a bag file you can replay the file just like a running system. Bags are a great tool for debugging and testing. You can treat a ROS bag like a recording of a running ROS system. When you play a bag file you can use most of the `ros2` CLI tools to inspect the recorded topics.

To replay the bag, first use `CTRL-C` to turn off the `turtlesim_node` and the `draw_square` node. Now in a new terminal replay the bag file using the following command:

```
$ ros2 bag play turtle1
[INFO] [rosbag2_storage]: Opened database 'turtle1'.
```

Nothing should happen visibly, but a lot is happening under the hood. To see what is happening go to a second terminal. Just like a running robot, you should be able to `list` and `echo` topics:

```
$ ros2 topic list
/parameter_events
/rosout
/turtle1/pose

$ ros2 bag info turtle1
x: 3.8595714569091797
y: 3.6481313705444336
theta: -1.2895503044128418
linear_velocity: 1.0
angular_velocity: 0.0
---
```

ROS2 Component Command

ROS2 Daemon Command

ROS2 Doctor Command

With any complex system problems can sometimes arise, and knowing how to describe your system and what is happening can do a lot to help others help you fix your problem. ROS 2 has a `doctor` command that you can use to print a variety of reports that you can use to help communicate the state of your system to others trying to provide help. Whether it is one of your co-workers, a vendor, or an on-line forum, providing detailed and complete information about your ROS system can go a long way to solving your problem. Let's call `--help` on the ROS 2 doctor command:

```
$ ros2 doctor --help
usage: ros2 doctor [-h] [--report | --report-failed] [--include-warnings]
```

Check ROS setup and other potential issues

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>--report, -r</code>	Print all reports.
<code>--report-failed, -rf</code>	Print reports of failed checks only.
<code>--include-warnings, -iw</code>	Include warnings as failed checks. Warnings are ignored by default.

As we can see from the help file we have a couple of report options. One option is to print the full report with `-r`, or just what failed with `-rf`. If you're running `ros2 doctor -r` you should see a fairly lengthy report generated giving information about your computer's operating system, your networking configuration, and your running ROS system. If you ever run into an issue you should always include this full report.

ROS 2 Interface

As you've already seen, ROS uses standard messages so that different packages and programs, potentially written in different programming languages, can all talk to one another. To make this all work, ROS uses standard messages, and communication protocols built on top of those standard messages. This can make finding type information about a particular message, service, or action difficult. To help developers write both CLI command calls and develop client code, the ROS CLI has the `interface` command. We've touched on this command briefly in other sections, as it is the go to tool for message type information.

To better understand the `interface` command let's start by looking at its high level help command to see what sub commands are available:

```
$ ros2 interface --help
usage: ros2 interface [-h]
                Call `ros2 interface <command> -h` for more detailed
                usage. ...
```

Show information about ROS interfaces

optional arguments:

`-h, --help` show this help message and exit

Commands:

<code>list</code>	List all interface types available
<code>package</code>	Output a list of available interface types within one package
<code>packages</code>	Output a list of packages that provide interfaces
<code>proto</code>	Output an interface prototype
<code>show</code>	Output the interface definition

Call ``ros2 interface <command> -h`` for more detailed usage.

The interface command are all geared towards helping you understand available message types. Let's examine the `list` sub command in depth. `list` will list all of the available messages, services, and actions on your system. This command has flags that can help you narrow down the scope of your search. Even a basic ROS installation has a lot of messages, so a tool you should get familiar with is `grep`. `grep` lets you search through some text to find what you are looking for quickly and easily. You can `grep` in a case insensitive manner using the `-i`

flag followed by the text you want to search for. We can tie this CLI tool to our interface tool by using the UNIX pipe operator `|`. The example below shows you how to use the list operation and then how to use it to search:

```
$ ros2 interface list --only-msgs
Messages:
  action_msgs/msg/GoalInfo
  action_msgs/msg/GoalStatus
  ... <DOZENS OF DIFFERENT TYPES> ...
  visualization_msgs/msg/MarkerArray
  visualization_msgs/msg/MenuEntry
$ ros2 interface list --only-msgs | grep -i point
geometry_msgs/msg/Point
geometry_msgs/msg/Point32
geometry_msgs/msg/PointStamped
map_msgs/msg/PointCloud2Update
pcl_msgs/msg/PointIndices
rcl_interfaces/msg/FloatingPointRange
sensor_msgs/msg/PointCloud
sensor_msgs/msg/PointCloud2
sensor_msgs/msg/PointField
trajectory_msgs/msg/JointTrajectoryPoint
```

Using `grep` to search through CLI output is a common tactic used by developers to find just the specific information they need. The next two sub commands `package` and `packages` can be used to first determine what ROS packages are on your system, and then to drill down into an individual package to determine what messages are in that package. Note that you can use `grep` just like before to search for your specific interest. The example below shows you how to first determine if `std_msgs` is installed and then to find out what sort of array types it contains:

```
$ ros2 interface packages
action_msgs
action_tutorials_interfaces
actionlib_msgs
builtin_interfaces
composition_interfaces
diagnostic_msgs
example_interfaces
geometry_msgs
lifecycle_msgs
logging_demo
map_msgs
nav_msgs
pcl_msgs
pendulum_msgs
rcl_interfaces
rosgraph_msgs
rqt_py_common
sensor_msgs
shape_msgs
std_msgs
std_srvs
stereo_msgs
tf2_msgs
trajectory_msgs
turtlesim
unique_identifier_msgs
visualization_msgs
kscottz@kscottz-ratnest:~/Code/ros2multirobotbook/src$ ros2 interface package
std_msgs | grep -i array
std_msgs/msg/Int8MultiArray
std_msgs/msg/Int32MultiArray
std_msgs/msg/MultiArrayLayout
std_msgs/msg/UInt64MultiArray
std_msgs/msg/Float32MultiArray
std_msgs/msg/UInt16MultiArray
std_msgs/msg/UInt32MultiArray
std_msgs/msg/Int16MultiArray
std_msgs/msg/ByteMultiArray
std_msgs/msg/Int64MultiArray
std_msgs/msg/Float64MultiArray
std_msgs/msg/UInt8MultiArray
std_msgs/msg/MultiArrayDimension
```

The next two commands are particularly helpful and you should remember them as they will make your life much easier. As we have discussed previously all message publication, service calls, and action calls in the CLI take in both the message type and data you want to transmit in YAML format. But what if you don't know the message format, and you don't know a lot about YAML? The `interface show` and `interface proto` commands make this process easier by respectively telling you first the message type and then the message format. Recall earlier in

the chapter when we called the `spawn` service on our turtle simulation. We can use `interface show` to tell us broadly about the service and what each of the values mean. We can then use `interface proto`, short for prototype, to then generate an empty message that we can fill out. See the example below:

```
$ ros2 interface show turtlesim/srv/Spawn
float32 x
float32 y
float32 theta
string name # Optional. A unique name will be created and returned if this is
empty
---
string name
$ ros2 interface proto turtlesim/srv/Spawn
"x: 0.0
y: 0.0
theta: 0.0
name: ''
"
$ ros2 service call /spawn turtlesim/srv/Spawn "{<copy and paste proto here>}"
```

You can see from the example above how handy these tools can be. It is worth noting that you need to paste the prototype into a set of quotes and curly braces for the call to work "`{<prototype>}`".

ROS 2 Launch

The `launch` command is used to run ROS launch files. Up until this point we've been running single ROS programs by hand using the `run` command, however this is not how larger ROS systems are generally operated and many robots command tens if not hundreds of small programs. The ROS launch command is different from most of the other ROS commands in that it has no sub commands and has a single function, to start a ROS launch file that executes multiple of programs. To illustrate this command let's take a look at its help file.

```
$ ros2 launch -h
usage: ros2 launch [-h] [-d] [-p | -s] [-a]
                  package_name [launch_file_name]
                  [launch_arguments [launch_arguments ...]]
```

Run a launch file

positional arguments:

package_name	Name of the ROS package which contains the launch file
launch_file_name	Name of the launch file
launch_arguments	Arguments to the launch file; '<name>:=<value>' (for duplicates, last one wins)

optional arguments:

-h, --help	show this help message and exit
-d, --debug	Put the launch system in debug mode, provides more verbose output.
-p, --print, --print-description	Print the launch description to the console without launching it.
-s, --show-args, --show-arguments	Show arguments that may be given to the launch file.
-a, --show-all-subprocesses-output	Show all launched subprocesses' output by overriding their output configuration using the <code>OVERRIDE_LAUNCH_PROCESS_OUTPUT</code> envvar.

Launch files are usually included with a ROS package and are commonly stored in a `launch` sub directory. Modern launch files are usually written in Python and end with the `*.launch.py` file extension. The `launch` command has two arguments, the first one is the package name and then the launch file name. If you are unaware of the launch files in your package you can use tab completion to list all the available launch files. Finally, some launch files have arguments that can be appended to the command. If you are unsure about what a launch file does, or what arguments it needs, the `--print` and `--show-args` commands will tell you this information. Let's read up on the `multisym.launch.py` launch file and then run it following the example below (end the simulation with `CTRL-C`):


```
$ ros2 launch turtlesim multisim.launch.py --show-args
```

Arguments (pass arguments as '<name>:=<value>'):

No arguments.

```
$ ros2 launch turtlesim multisim.launch.py --print
```

```
<launch.launch_description.LaunchDescription object at 0x7f75aab63828>
```

```
└─ ExecuteProcess(cmd=[ExecInPkg(pkg='turtlesim', exec='turtlesim_node'), '--ros-args'], cwd=None, env=None, shell=False)
```

```
└─ ExecuteProcess(cmd=[ExecInPkg(pkg='turtlesim', exec='turtlesim_node'), '--ros-args'], cwd=None, env=None, shell=False)
```

```
$ ros2 launch turtlesim multisim.launch.py
```

```
[INFO] [launch]: All log files can be found below /home/kscottz/.ros/log/2020-06-24-14-39-03-312667-kscottz-ratnest-20933
```

```
[INFO] [launch]: Default logging verbosity is set to INFO
```

```
[INFO] [turtlesim_node-1]: process started with pid [20944]
```

```
[INFO] [turtlesim_node-2]: process started with pid [20945]
```

```
^C[WARNING] [launch]: user interrupted with ctrl-c (SIGINT)
```

ROS 2 Lifecycle

ROS 2 has a new feature called `lifecycle` which allows for greater control over the state of a ROS node. Roughly, this feature allows nodes to have complex start-up and shut-down procedures that are correctly handed. An example of such a node would be one that controls a sensor or actuator that needs to perform a power-on self-test or calibration procedure prior to running. [The ROS design docs](#) give a great primer on the states and transitions in a lifecycle node. Let's look at the `lifecycle` command to determine what sub commands are available:

```
$ ros2 lifecycle -h
```

```
usage: ros2 lifecycle [-h]
```

```
Call `ros2 lifecycle <command> -h` for more detailed
usage. ...
```

Various lifecycle related sub-commands

optional arguments:

```
-h, --help            show this help message and exit
```

Commands:

```
get    Get lifecycle state for one or more nodes
list   Output a list of available transitions
nodes  Output a list of nodes with lifecycle
set    Trigger lifecycle state transition
```

Call `ros2 lifecycle <command> -h` for more detailed usage.

The `nodes` sub command will list all the lifecycle nodes on a given system. Once you have listed the nodes, you can list each node's available transitions using `ros2 lifecycle list <nodename>`. These transitions are dictated by the node's current state with some states having more transitions available than others. If instead of available transitions you wish to query the current state you can use `lifecycle get` to return the current state of your target node. Once you have satisfactorily determined the state of the node and the available transitions, the `lifecycle set` command can be used to trigger the node to transition to a new state. Generally, these CLI commands are used to diagnose failure modes of systems, or to manually transition a particular component.

ROS 2 msg (Message)

ROS 2 Eloquent is the last version of ROS to use the `msg` command. All of the commands in `msg` are mirrored in the `interface` command. These features are currently deprecated and will be removed in Foxy.

ROS 2 pkg (Package)

The ROS 2 package command is a very useful command to understand what ROS packages are installed on your system, where they are installed, and the executables contained within each package. These tools are particularly useful for understanding an existing robot configuration and finding tools that are only used on occasion. Let's start by taking a look at the help file for the `pkg` command:

```
$ ros2 pkg -h
usage: ros2 pkg [-h] Call `ros2 pkg <command> -h` for more detailed usage. ...
```

Various package related sub-commands

optional arguments:

`-h, --help` show this help message and exit

Commands:

<code>create</code>	Create a new ROS2 package
<code>executables</code>	Output a list of package specific executables
<code>list</code>	Output a list of available packages
<code>prefix</code>	Output the prefix path of a package
<code>xml</code>	Output the XML of the package manifest or a specific tag

Call ``ros2 pkg <command> -h`` for more detailed usage.